Mesh Refinement and Multigrid Self-Gravity in the AthenaK Shearing Box

Implementation Document

Phase 0 (refinement infrastructure), Phase 1 (shear-periodic multigrid boundaries), Phase 2 (gravity on refined meshes), Phase 3 (the stratified box), Phase 4 (ideal MHD with self-gravity)

`athenak-multigrid` tree, branch `multigrid`

commit range `cc4404ba..fef5c6c9` (plus the Phase-0a predecessors `e7ef37f2`, `cc4404ba`)

August 18, 2026 (Phase 4: October 26–27, 2026)

Abstract

This document describes *how* mesh refinement and the multigrid Poisson solver were made to work in the AthenaK shearing box — the data structures, algorithms, call sites, invariants, and guards — as a companion to `multigrid_selfgravity_validation.pdf`, which records *what the resulting code does* on test problems. Nothing here is a result; everything here is a mechanism.

The work divides into three phases. **Phase 0** lifts AthenaK's blanket prohibition on shearing box + refinement, replacing it with an enforced refinement *policy* (uniform level on both shear-periodic x_1 faces; complete azimuthal rings when orbital advection is active), makes orbital advection work level by level, adds a fully consistent non-FARGO (full-velocity) mode, and rebuilds all shear-boundary communication state after every adaptive mesh update. **Phase 1** makes the multigrid solver shear-aware by treating shear-periodicity as what it is — a boundary *identification* with a time-dependent azimuthal offset — so the 7-point Laplacian, smoother, and transfer operators are untouched and only the x_1 ghost fill changes: a gather/remap/scatter through per-level global boundary planes, using the same conservative remap-flux kernels as orbital advection. **Phase 2** carries that fill onto refined meshes: interior rings (2a) need only a guard relaxation, refined *boundary* annuli (2b) require the same algorithm re-expressed at multigrid-octet granularity in host code, and adaptive refinement (2c) requires every shear structure sized at construction to become rebuildable, which it does by riding the solver's existing remesh-detection path.

A final section documents the rank-consistency fix (`ShearingBox::ConsistentDx2`) that removed a deterministic MPI message-size desynchronization in the shear exchange, found at 64 ranks.

Contents

1 Scope, sources, and how to read this **4**

1.1 What this document is . . . 4

1.2 Commit provenance . . . 4

1.3 Files touched . . . 5

2 Background: what the shearing box already had **6**

3 Phase 0: mesh refinement in the shearing box **7**
3.1 The two invariants, and what they protect . . . 7
3.2 Where the policy is enforced . . . 7
3.3 The graded refinement cap . . . 8
3.4 Ring-wise flag synchronization . . . 9
3.5 The derefinement-ordering fix . . . 9
3.6 Non-FARGO (full-velocity) mode . . . 9
3.7 The shift-depth estimate and its runtime guard . . . 10
3.8 Phase 0c: making the boundary state rebuildable under AMR . . . 10
3.9 A user AMR criterion for testing: the moving ring . . . 11

4 Phase 1: shear-periodic boundaries for the multigrid solver **11**
4.1 The design decision . . . 11
4.2 The shear phase $q\Omega_0 t_s$ . . . 12
4.3 Data structures . . . 12
4.4 The block-level fill: gather, remap, scatter . . . 12
4.5 The root-grid fill . . . 13
4.6 Hooks into the existing solver . . . 14
4.7 Two corrections uncovered by the plumbing . . . 14
4.8 Driver-independent task-list execution . . . 14
4.9 The MPI seam . . . 15
4.10 Phase-1 guards and parameters . . . 15

5 Phase 2: multigrid gravity on refined shearing-box meshes **15**
5.1 Phase 2a: interior rings . . . 15
5.2 Phase 2b: sheared octet ghosts . . . 16
5.3 Phase 2c: adaptive refinement . . . 17
5.4 Phase 2d: transverse slopes of the fine ghosts at level jumps . . . 17
5.5 Phase 2e: quadratic level-jump ghosts (`mg_fc_interp`) . . . 19
5.6 End-to-end control flow . . . 20

6 Phase 3: the stratified box — vertical gravity, slab-open multigrid boundaries, and cooling **20**
6.1 Phase 3a: hydro-side stratification terms . . . 20
6.2 Phase 3b: slab-open vertical boundaries for the multigrid solver . . . 21
6.3 Phase 3 validation . . . 23
6.4 Phase 3c: the SC14 problem generator . . . 24

7 Phase 4: ideal MHD with self-gravity **24**
7.1 Phase 4a: the gravito-turbulent box with a field . . . 25
7.2 Phase 4b: the magneto-Jeans problem generator . . . 26
7.3 Phase 4c: the swing problem generator with a field . . . 27
7.4 Phase 4: what is not there yet . . . 28

8 Rank-consistent integer shifts **29**

9 Input-file interface **30**
9.1 Parameters 30
9.2 Reference inputs 31
9.3 Building 31

10 Guard reference **31**

11 Known limitations and deferred work **32**

1 Scope, sources, and how to read this

1.1 What this document is

This is an implementation reference. For each phase it answers: what invariant does the code rely on, where is that invariant established and enforced, what data structure carries the new state, which function performs the work, and from which call site is it reached. Numerical results, convergence rates, and comparisons against the FFT solver live in the companion validation document and notebook:

`validation/multigrid_selfgravity_validation.pdf`	results, tests, figures
`validation/multigrid_validation_tests.ipynb`	the analysis behind them
`validation/fft_selfgravity_validation.pdf`	the FFT solver used as oracle

1.2 Commit provenance

The implementation commits, in order, are listed in Table 1. Commits that only touch documentation, notebooks, figures, or input files are omitted.

Commit	Phase	Content
`e7ef37f2`	0a	Refinement policy, per-level FARGO, non-FARGO mode
`cc4404ba`	0b	Uniformly refined x_1 boundary faces
`f0acd383`	0c	Derefinement-ordering fix (whole-list sort)
`b5388c02`	0c	AMR: rebuild x_1-boundary state per mesh update; graded cap
`e853a004`	1	`multigrid_shear.cpp`; shear-periodic MG boundaries
`be32edce`	1	Multi-rank shear fill (one `Allreduce` seam)
`d9eacaa2`	—	Tomida & Stone (2023) consistency tests; MG driver setters
`0cd084d2`	2a	Gravity on interior-ring refined meshes (guard relaxation)
`a214fddf`	2b	Sheared octet ghosts; scalar remap-flux twins
`008cb5a2`	2c	AMR + multigrid gravity (rebuild on remesh)
`89904070`	—	Rank-desync fix: `ConsistentDx2`, sender and unpack
`ca63f727`	—	Same fix, recv-posting sites (completes all seven)
`e1e0a072`	2d	Centred transverse slopes in the fine ghosts at level jumps
`d86c16ac`	2d	The same, on `fargo-amr-mhd`
`57a70d8d`	2d	`mg_interface_wave` test problem (`fargo-amr-mhd`)
`aad3deb7`	2e	Quadratic level-jump ghosts (`mg_fc_interp`)
`917b93bc`	2e	Quadratic pair on the finest MG level only
`5808b2fd`	2e	Comment fix (the same three on `fargo-amr-mhd`: `ddae4ca6`, `64a9e22e`, `c9eb45e2`)

Table 1: Implementation commits covered by this document.

1.3 Files touched

File	Role
New	
`src/multigrid/multigrid_shear.cpp`	The entire shear-periodic multigrid boundary implementation (458 lines): `ComputeShearQomt`, `ExecuteMGTaskList`, `AllocateShearPlanes`, `FillShearGhostPack`, `FillShearOctetGhosts`, `RootShearBoundaryX1`
Modified — mesh / refinement	
`src/mesh/mesh.cpp`	Prohibition replaced by policy warning
`src/mesh/build_tree.cpp`	`Mesh::CheckShearingBoxRefinement`
`src/mesh/mesh.hpp`	Declaration
`src/mesh/mesh_refinement.cpp`	Boundary veto, graded cap, ring-wise flag sync, post-update policy check, shear reinit hook, derefinement sort fix
`src/mesh/mesh_refinement.hpp`	`shearing_box_`, `sbox_ring_policy_`
Modified — shearing box	
`src/shearing_box/shearing_box.{cpp,hpp}`	`SetX1BndryMBs`, `ReinitAfterMeshUpdate`, `ConsistentDx2`, `orbital_advection` flag
`src/shearing_box/shearing_box_cc.cpp`	`AllocateBuffers`, non-FARGO azimuthal offsets, consistent `joffset`
`src/shearing_box/shearing_box_fc.cpp`	`AllocateBuffers`, consistent `joffset`
`src/shearing_box/shearing_box_tasks.cpp`	Consistent `joffset` at the three recv/clear sites
`src/shearing_box/shearing_box_srcterms.cpp`	Full-velocity source terms
`src/shearing_box/orbital_advection.cpp`	`maxjshift` estimate
`src/shearing_box/orbital_advection_cc.cpp`	Per-step shift guard
`src/shearing_box/remap_fluxes.hpp`	Scalar per-face remap twins
Modified — gravity / multigrid	
`src/gravity/mg_gravity.cpp`	Shear enablement, guards, remap-order parameter, mean-source subtraction, phase freeze
`src/multigrid/multigrid.hpp`	Shear members and declarations
`src/multigrid/multigrid.cpp`	Plane deallocation
`src/multigrid/multigrid_driver.cpp`	BC plumbing, octet wrap, octet face flags, octet fill hooks, root fill hook, AMR rebuild
`src/multigrid/multigrid_tasks.cpp`	Block-level fill hook in `PhysicalBoundary`
Modified — physics modules and pgens	
`src/hydro/hydro.cpp`, `src/mhd/mhd.cpp`	Optional orbital advection; MHD+FARGO+refinement fatal
`src/pgen/tests/shwave.cpp`, `swing.cpp`	Background shear in ICs; `MovingRingRefine` user AMR criterion
`src/CMakeLists.txt`	`multigrid/multigrid_shear.cpp`

2 Background: what the shearing box already had

The local shearing-box approximation solves the fluid equations in a frame corotating at Ω_0 at radius R_0, with background flow $\mathbf{v}_{\rm bg} = -q\Omega_0 x\,\hat{\mathbf{e}}_y$. The x_1 boundaries are *shear-periodic*: the domain is identified with its images displaced azimuthally at the shear rate,

$$u(x + L_x,\, y,\, z,\, t) \;=\; u(x,\; y + q\Omega_0 L_x t,\; z,\; t)\,, \tag{1}$$

which is a *boundary identification*, not a change of the interior equations. This single observation is the organizing idea of Phases 1 and 2.

AthenaK ships three pieces of pre-existing machinery that this work builds on.

The shear-periodic boundary exchange (`shearing_box_cc.cpp`, `shearing_box_fc.cpp`). `ShearingBoxCC` and `ShearingBoxFC` derive from `ShearingBox`. Each holds a list of the MeshBlocks on this rank that touch the $\pm x_1$ faces (`x1bndry_mbgid`), send/recv buffers, and MPI request arrays. The exchange sends each boundary block's ghost data to up to three azimuthal targets, chosen from the integer part of the shift, and applies the fractional part with a conservative remap.

Orbital advection / FARGO (`orbital_advection*.cpp`). In the default (shear-subtracted) frame, velocities are perturbations about $\mathbf{v}_{\rm bg}$ and the background advection is applied analytically: an integer cell shift by communication with x_2-face neighbors, plus a conservative fractional remap. The communicated buffer depth is `ng` + `maxjshift`.

Conservative remap fluxes (`remap_fluxes.hpp`). `DC_RemapFlx`, `PLM_RemapFlx`, `PPMX_RemapFlx` are Kokkos *team* kernels that, given a padded scratch row q and a fractional shift $\epsilon \in (-1, 1)$, write the remap flux at each cell face. A row is shifted by

$$q_j^{\rm new} \;=\; q_j - (F_{j+1} - F_j)\,, \tag{2}$$

which is conservative by construction. The same kernels serve orbital advection, the shear-periodic exchange, the FFT gravity solver, and (Phase 1) the multigrid solver; one shared implementation means the shift used by gravity and the shift used by the hydro solver cannot silently diverge.

The multigrid solver (`src/multigrid/`). An FAS multigrid Poisson solver, ported from Athena++ (Tomida & Stone 2023). Its hierarchy has three regimes, which matters greatly for Phase 2:

1. **Block levels**: one `Multigrid` object over the `MeshBlockPack`, coarsened down to one cell per block. Device data, per-block, communicated with boundary buffers.
2. **Octet levels**: below the block levels, on refined meshes, the mesh is represented as 2^3 *octets* per level. These live in *host* memory and are *replicated on every rank* (via an `Allgatherv` in the block→root transfer).
3. **Root grid**: a single global array, also replicated per rank, coarsened to the coarsest level and solved there.

On uniform grids there are no octet levels. Phase 1 therefore touches regimes 1 and 3; Phase 2b exists precisely because boundary refinement creates regime-2 objects sitting on the shear faces.

3 Phase 0: mesh refinement in the shearing box

Before this work, `mesh.cpp` contained an unconditional fatal:

```
// FIXME: The shearing box is not currently compatible with SMR/AMR
if (multilevel && pin->DoesBlockExist("shearing_box")) { ... std::exit(EXIT_FAILURE); }
```

Phase 0 replaces the prohibition with a *policy*: refinement configurations that preserve the assumptions of the shear machinery are allowed, everything else aborts with a message naming the violation. The policy is not a convenience; each clause corresponds to a specific assumption in existing code.

3.1 The two invariants, and what they protect

Invariant I1 (uniform boundary level). All MeshBlocks touching either shear-periodic x_1 face share *one* refinement level. That level may exceed the root level, provided both faces are refined uniformly over their full x_2/x_3 extent.

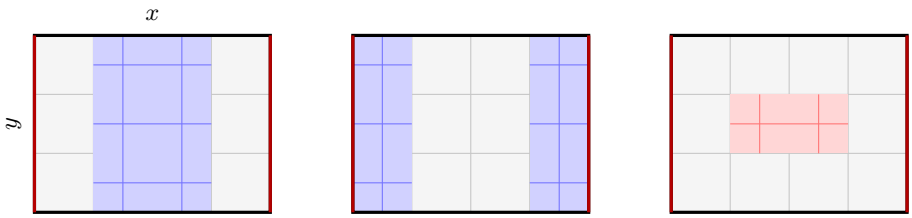
What it protects. A code audit (commit `cc4404ba`) established that nothing in the shear machinery references the root grid as such. The unshifted wrap across x_1 goes through the ordinary neighbor machinery — the mesh tree treats `shear_periodic` exactly like `periodic`, including 2:1 nesting — and the shear pass redistributes ghost data azimuthally *within* each face using per-block Δx_2 arithmetic and a level-aware `FindTargetMB`. Its real assumptions are only: (i) same-size blocks along each face, and (ii) a same-level wrap between the two faces. Refining both faces uniformly satisfies both, and is also physically natural — a structure at the shear boundary has images on *both* faces. Mixed-level boundaries remain fatal because combining the y-shift with prolongation inside the shear communication is not implemented.

Invariant I2 (annular rings, FARGO only). With `orbital_advection = true`, every refined region must span the full x_2 extent: for each (ℓ, l_{x1}, l_{x3}) the number of leaf blocks must equal $N^{\mathrm{root}}_{\mathrm{mb},x2} \ll (\ell - \ell_{\mathrm{root}})$.

What it protects. Orbital advection communicates only with x_2-face neighbors and applies a per-block integer shift. If a refined patch ended partway around the azimuth, a fine block would have a coarse x_2-face neighbor, and the per-block shift kernel plus the existing (same-size) communication pattern would both be invalid. Requiring complete rings makes every x_2-face neighbor same-level and same-size, so the existing pattern remains valid *level by level* with no new code. Without FARGO the constraint is unnecessary and is not applied, which is why the `orbital_advection` toggle (§3.6) is part of Phase 0 rather than an unrelated convenience.

3.2 Where the policy is enforced

Enforcement is deliberately layered: a *veto* that prevents most violations from being requested, and a *fatal check* that catches anything that slips through.



(a) interior ring **allowed** (b) boundary annuli **allowed** (c) partial ring **fatal** (I2)

Figure 1: Refinement policy in the x–y plane. Red edges are the shear-periodic x_1 faces. (a) Interior ring: boundary at root level, refinement spans the full azimuth. (b) Both faces uniformly refined — allowed by I1 from Phase 0b onward, and the configuration that creates boundary octets in Phase 2b. (c) A patch that does not close in y violates I2 whenever orbital advection is on.

Site	Kind	Effect
`Mesh::BuildTreeFromScratch`	fatal	I1, I2 checked on the freshly built tree
`Mesh::BuildTreeFromRestart`	fatal	Same, on restart
`MeshRefinement::CheckForRefinement`	veto	Clears refine flags that would violate I1 (boundary blocks; graded cap)
`MeshRefinement::CheckForRefinement`	sync	Ring-wise OR/AND of flags to enforce I2 by construction
`MeshRefinement::AdaptiveMeshRefinement`	fatal	I1, I2 re-checked after `RedistAndRefineMeshBlocks`

`Mesh::CheckShearingBoxRefinement(ParameterInput*)` (`build_tree.cpp`) is the single implementation of both invariants. It returns immediately unless the mesh is multilevel and a `<shearing_box>` block exists. For I1 it scans `lloc_eachmb` for blocks with $l_{x1} = 0$ or $l_{x1} = N_{\mathrm{mb},x1}(\ell) - 1$ and requires $\ell_{\min} = \ell_{\max}$ over that set. For I2 it accumulates a `std::map` keyed on (ℓ, l_{x1}, l_{x3}) counting leaf blocks per ring and requires each count to equal the full azimuthal block count at that level. On a violation it prints the offending ring and dumps every refined block's logical location, which is what makes a failed AMR configuration diagnosable at all.

3.3 The graded refinement cap

Vetoing refinement of blocks that *sit* on the boundary is not enough: 2:1 balancing in `UpdateMeshBlockTree` propagates refinement outward from a newly refined block, and can reach the boundary columns from a few blocks away. Before commit `b5388c02` this turned an over-eager AMR criterion into a fatal abort mid-run.

The cap is a geometric room requirement. Let ℓ_b be the (uniform, by I1) boundary level and consider a block at level ℓ with $s = \ell - \ell_b \geq 1$ requesting refinement to $\ell + 1$. Balancing then requires a buffer column of every intermediate level between it and the boundary, of total width $(1 - 2^{-s})$ boundary-level columns on each side — and, expressed in integer logical coordinates *at the block's own level*,

$$2^{s+1} - 1 \;\leq\; l_{x1} \;\leq\; N_{\mathrm{mb},x1}(\ell) - 2^{s+1}. \tag{3}$$

The implementation reads exactly that:

```
if (lx1 == 0 || lx1 == (nmbx1-1)) {          // MB on boundary: never refine
  refine_flag.h_view(m+mbs) = 0;
} else if (lev >= (blev+1)) {                // graded cap on interior MBs
```

```
  std::int32_t gap = (1 << ((lev-blev)+1));
  if ((lx1 < (gap-1)) || (lx1 > (nmbx1-gap))) { refine_flag.h_view(m+mbs) = 0; }
}
```

With the cap in place, any I1 violation surviving to the post-update check is a genuine bug rather than a criterion asking for too much — which is what makes it reasonable to keep that check fatal.

3.4 Ring-wise flag synchronization

I2 is enforced *constructively* rather than by veto, immediately after the global flag `Allgatherv` in `CheckForRefinement`. Flags are collapsed into a map keyed on (ℓ, l_{x1}, l_{x3}) with the rule

$$\text{refine if } \textit{any} \text{ member is flagged;} \quad \text{derefine only if } \textit{every} \text{ member is flagged,} \tag{4}$$

and then written back to all members of the ring. The "any refine wins" direction guarantees that a criterion that fires anywhere in a ring produces a complete ring; the unanimity requirement for derefinement prevents a ring from being partially destroyed.

The ordering matters: this runs *after* the `Allgatherv`, on the globally consistent flag array, so every rank computes the identical map and the result is deterministic without any additional communication.

3.5 The derefinement-ordering fix

Ring-complete derefinement exposed a latent upstream bug. `UpdateMeshBlockTree` sorts the derefinement-candidate list finest-first so that `MeshBlockTree::Derefine`'s finer-neighbor veto operates on an already-updated tree, but used `&cllderef[ctnd-1]` as the end iterator — excluding the last element:

```
- std::sort(cllderef, &(cllderef[ctnd-1]), Mesh::GreaterLevel);
+ std::sort(cllderef, &(cllderef[ctnd]),   Mesh::GreaterLevel);
```

When a mixed-level derefinement left a fine-level parent in the unsorted last slot, coarser parents merging before it were vetoed *in an order-dependent way*: some parents of an annulus merged and some did not, splitting what should have been a ring-complete derefinement and tripping the I2 fatal. Outside the shearing box the bug only delayed some derefinements by one AMR interval, which is why it had gone unnoticed.

3.6 Non-FARGO (full-velocity) mode

`<shearing_box> orbital_advection` (default `true`) selects the frame. With `false`, `OrbitalAdvectionCC/FC` are simply not constructed (`porb_u = porb_b = nullptr`) while the shear-periodic BCs and the Coriolis/tidal source terms stay active, and the solver integrates the full shear flow directly. This mode exists for two reasons: it removes constraint I2 entirely (useful for arbitrary refinement patterns), and it provides an independent frame against which the FARGO path can be checked.

Consistency requires three coordinated changes.

(i) Source terms (`shearing_box_srcterms.cpp`). The shear-subtracted frame uses the reduced Coriolis coefficient $(2-q)\Omega_0$ on $\dot{m}_2$ because the background shear's contribution is carried analytically by orbital advection. In the full-velocity frame the velocities *include* the background, so the code applies the full Coriolis force plus the radial tidal force:

$$\dot{m}_1 \mathrel{+}= 2\Omega_0 m_2, \qquad \dot{m}_2 \mathrel{-}= 2\Omega_0 m_1, \qquad \dot{m}_1 \mathrel{+}= 2q\Omega_0^2 \rho\, x, \tag{5}$$

with the corresponding tidal work added to the total energy for an ideal EOS.

(ii) Boundary momentum/energy offsets (`shearing_box_cc.cpp`). The background azimuthal velocity jumps by $q\Omega_0 L_x$ across the box, so data wrapped through the shear-periodic boundaries must be offset to represent the *local* frame. After the unpack, for each ghost cell:

$$E \mathrel{+}= \delta v_y\, m_2 + \tfrac{1}{2}\rho\, \delta v_y^2, \qquad m_2 \mathrel{+}= \rho\, \delta v_y, \qquad \delta v_y = \begin{cases} +q\Omega_0 L_x & \text{at } ix_1 \\ -q\Omega_0 L_x & \text{at } ox_1 \end{cases} \tag{6}$$

Note the order: energy uses the pre-update m_2, so the momentum update must come second.

(iii) Initial conditions (`shwave.cpp`, `swing.cpp`). Every problem generator branch adds $v_y^{(0)} = -q\Omega_0 x$ to the initial azimuthal velocity when `orbital_advection = false`, with the matching kinetic-energy contribution.

MHD restriction. MHD orbital advection remaps face-centered B through EMFs; the correctness of that remap at fine/coarse interfaces (specifically, $\nabla\cdot B$ preservation) has not been analyzed, so MHD + FARGO + refinement remains fatal in `MHD::MHD`. Hydro is supported, and MHD without orbital advection is supported.

3.7 The shift-depth estimate and its runtime guard

Orbital advection communicates a buffer of depth `ng` + `maxjshift`, where `maxjshift` is estimated once at construction. The original estimate, $\lceil \mathrm{CFL}\cdot \max|x_1| \rceil$, omitted the $q\Omega_0$ factor. It was corrected to

$$\texttt{maxjshift} = \lfloor \mathrm{CFL}\,(q\Omega_0) \max|x_1| \rfloor + 1, \qquad \text{doubled on multilevel meshes,} \tag{7}$$

the doubling being headroom for fine blocks at large $|x_1|$, whose shift in their own (smaller) cell units is larger whenever Δt is not set by that same region.

Because that is still a heuristic, `OrbitalAdvectionCC::RecvAndUnpackCC` now *verifies* it every step against this step's Δt and each block's own geometry, before unpacking:

$$\max_m \left\lfloor \frac{q\Omega_0 \Delta t\, \max|x_1|_m}{\Delta x_{2,m}} \right\rfloor \;\le\; \texttt{maxjshift}, \tag{8}$$

aborting with a message that names the observed shift and the buffer depth. This converts a silent out-of-bounds read into a diagnosable failure.

3.8 Phase 0c: making the boundary state rebuildable under AMR

AMR renumbers MeshBlock GIDs and reassigns blocks to ranks on every update, but `x1bndry_mbgid`, the MPI request arrays, and the send/recv buffers were all built once in the `ShearingBox` constructor. Running AMR would have silently corrupted the shear exchange. The restructuring:

`ShearingBox::SetX1BndryMBs()`	List and request-array construction, extracted from the constructor and re-expressed against `pmy_pack` so it can run on the current pack. Grow-only `realloc` of `x1bndry_mbgid`; all requests reset to `MPI_REQUEST_NULL`.
`ShearingBox::AllocateBuffers()`	New pure virtual; `ShearingBoxCC` and `ShearingBoxFC` override it with the grow-only buffer reallocation extracted from their constructors. `ShearingBoxCC` now stores `nvar` because it needs it to size buffers outside the constructor.
`ShearingBox::ReinitAfterMeshUpdate()`	`SetX1BndryMBs()` followed by the virtual `AllocateBuffers()`.

The call site is `MeshRefinement::AdaptiveMeshRefinement`, placed after `RedistAndRefineMeshBlocks` and the post-update policy check, and *before* `InitBoundaryValuesAndPrimitives` — i.e. after the block metadata is rebuilt and before any boundary exchange touches the new mesh. It fires for `phydro->psbox_u`, `pmhd->psbox_u`, and `pmhd->psbox_b` when present.

The communicator is created once (`MPI_Comm_dup` in the constructor) and not re-duplicated; only the request arrays and buffers are rebuilt. Buffers grow but never shrink, which avoids reallocation churn when the block count oscillates; the extra slots are simply unused.

3.9 A user AMR criterion for testing: the moving ring

Exercising ring creation and destruction requires a criterion whose behavior is known in advance. `MovingRingRefine` (in `shwave.cpp`; an identical `SwingMovingRingRefine` in `swing.cpp`) refines every block whose x_1 extent overlaps

$$[x_c(t) - w,\ x_c(t) + w]\,, \qquad x_c(t) = x_{c0} + x_{\rm amp}\sin(\omega_r t), \tag{9}$$

and flags all others for derefinement. Because the criterion depends only on x_1 it naturally flags complete rings; the ring sync and boundary vetoes of §3.2 supply the rest of the policy. It runs on host data (block bounds only, no kernel) and is enrolled from the problem generator on both fresh starts and restarts, controlled by `<problem> amr_moving_ring` and the parameters `ring_xc0`, `ring_xamp`, `ring_freq`, `ring_hwidth`.

Two practical consequences of user-criterion AMR are worth recording, because they constrain what tests can prove: a run starts on the *unrefined* root mesh and refines within the first `refinement_interval` cycles, so bit-identity with an equivalent SMR run is structurally impossible; and `max_nmb_per_rank` must carry transient headroom for the moment when the old ring has not yet been destroyed and the new one already exists.

4 Phase 1: shear-periodic boundaries for the multigrid solver

4.1 The design decision

Equation (1) is an identification of the domain with its azimuthally displaced image. It changes *which cells are neighbors*, not *what the operator is*. So:

Design rule. The 7-point Laplacian, the smoother, the restriction and prolongation operators, and the coarse-grid solve are untouched. Shear-periodicity enters exclusively as a modified x_1 ghost fill — one that shifts the incoming plane by $\pm q\Omega_0 t_s L_x$ in y — applied identically at every level of the multigrid hierarchy.

Two properties of multigrid make the "identically at every level" clause cheap. First, the FAS V-cycle's fixed point satisfies the *finest*-level system; the coarse levels only accelerate convergence. Coarse-level remap quality therefore affects the convergence *rate*, not the answer. Second, because the shift is generally a non-integer number of cells at *every* level anyway, no level is a special case: one code path serves all of them.

4.2 The shear phase $q\Omega_0 t_s$

The box is strictly shear-periodic at times $t_n = nL_y/(q\Omega_0 L_x)$; measuring the offset from the most recent such instant keeps the shift — and hence the remap error — minimal:

```
Real MultigridDriver::ComputeShearQomt(Real time) const {
  Real tshear = ly/(mg_qshear_*mg_omega0_*lx);
  Real dts = time - std::floor(time/tshear)*tshear;
  return mg_qshear_*mg_omega0_*dts;
}
```

`MGGravityDriver::Solve` freezes `mg_qomt_` once per solve, from `pmesh->time`. This is deliberately the *same convention as the FFT solver*, so both solvers see identical boundary identifications at any instant and can be compared bit-for-bit. Note that hydro's own shear-periodic BC uses `time+dt` on the final RK stage; the resulting $O(\Delta t)$ offset between gravity and hydro is a pre-existing, FFT-validated convention and is intentionally *not* "fixed" in the multigrid path, since doing so would break the solver-to-solver comparison that validates it.

4.3 Data structures

Two members are added to `Multigrid` (`multigrid.hpp`):

`DvceArray4D<Real> *shear_plane_`	One global boundary plane per multigrid level, dimensioned `(2, nvar*ngh, gnz, gny)`. Index 0 selects the face: plane 0 is the *source for the inner ghosts* (gathered at the outer boundary), plane 1 the source for the outer ghosts.
`DvceArray2D<int> shear_goffs_`	Per-block global (y, z) cell offsets at the finest level, $(l_{x2} n_{x2},\ l_{x3} n_{x3})$, shifted right by the level index at use.
`int shear_reflev_`	Refinement offset of the x_1-boundary blocks above root (Phase 2b; zero on uniform grids and interior-only refinement).

and four on `MultigridDriver`: `mg_shear_enabled_`, `mg_qshear_`/`mg_omega0_`, `mg_qomt_` (frozen per solve), and `mg_remap_order_`.

Plane extents at level l (with $\texttt{ll} = \texttt{nlevel_} - 1 - l$) are

$$\texttt{gny} = (N_{\mathrm{mb},x2} \ll \texttt{shear_reflev_})\,(n_{x2} \gg \texttt{ll})\,, \qquad \texttt{gnz} = (N_{\mathrm{mb},x3} \ll \texttt{shear_reflev_})\,(n_{x3} \gg \texttt{ll})\,, \tag{10}$$

i.e. the full y–z extent of the mesh at the boundary blocks' level, coarsened to this multigrid level.

4.4 The block-level fill: gather, remap, scatter

`Multigrid::FillShearGhostPack(qomt, order)` is three kernels submitted on one execution-space instance — stream-ordered, so no explicit fences are required between them.

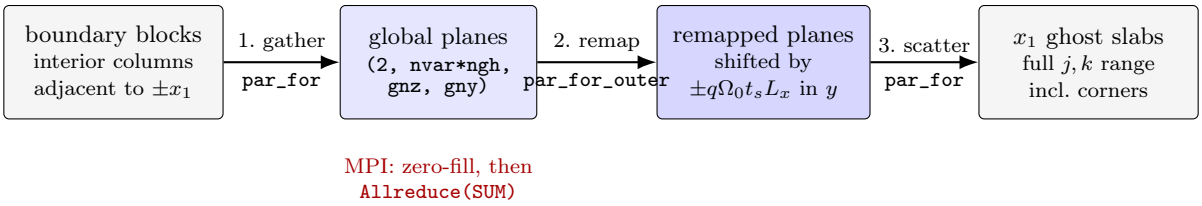


Figure 2: The block-level shear ghost fill. The only MPI seam is a single `Allreduce` between gather and remap.

1. Gather. Over all blocks and all interior (k, j), blocks whose `mb_bcs` carries `shear_periodic` on an x_1 face write their `ngh` interior columns adjacent to that face into the plane at global index $(\mathtt{gk}, \mathtt{gj})$ derived from `shear_goffs_`. Note the crossing: an *outer*-boundary block feeds plane 0, which will fill *inner* ghosts. Non-boundary blocks — including all interior fine blocks under Phase 2a — are skipped by the `mb_bcs` test, with no extra logic.

2. Remap. A team kernel over (face × var × depth, `gk`). For each row, the shift $y_{\rm sh} = \pm\, q\Omega_0 t_s L_x$ is split:

$$\mathtt{joffset} = \lfloor y_{\rm sh}/\Delta y_l \rfloor\,, \qquad \epsilon = \mathrm{fmod}(y_{\rm sh}, \Delta y_l)/\Delta y_l. \tag{11}$$

The integer part is folded into a *periodic-wrapped scratch load* $q_{jf} = \mathrm{plane}[(jf - \mathtt{PAD} - \mathtt{joffset}) \bmod \mathtt{gny}]$, and the fractional part is applied by the shared conservative kernels via Eq. (2). Because a full period is loaded before any write-back, the remap is safely in-place.

The scratch padding is `PAD = 3`: PPMX reads $j-2 \ldots j+2$ plus the face at $ju+1$, so three is exactly sufficient — and, crucially, independent of the shift size, because the integer part never enters the stencil. This is why $\mathtt{ngh} = 1$ imposes no stencil limitation on the multigrid solver even though the shift can be many cells.

3. Scatter. Over the *full* slab $k, j \in [0, \mathtt{ncells} + 2\mathtt{ngh})$ — not just interior cells — with periodic wrap in y and z. Writing the slab's edge and corner ghosts matters: trilinear prolongation reads them. The read set (interior columns) and write set (ghost columns) are disjoint, so gather and scatter cannot race.

4.5 The root-grid fill

The root grid is a single replicated global array, so it needs no planes and is remapped in place by `MultigridDriver::RootShearBoundaryX1`, a single team kernel that loads the source column with the wrapped offset, applies the same remap fluxes, and writes the destination ghost column. It fills the interior-row ghosts only; the x_2/x_3 passes of `MGRootBoundary` that follow supply the ghost-corner rows by periodic wrap of these already-remapped rows. At $q\Omega_0 t_s = 0$ it reduces exactly to the plain periodic copy, because the remap fluxes vanish identically at $\epsilon = 0$ — which is what makes the "shear-periodic instant equals periodic, bitwise" test meaningful.

4.6 Hooks into the existing solver

Call site (file)	What it does
`MultigridDriver::MultigridDriver` `multigrid_driver.cpp`	`shear_periodic` is kept as its own `mg_mesh_bcs_` value instead of collapsing to `mg_zerofixed`
`MGGravityDriver::MGGravityDriver` `mg_gravity.cpp`	Enables shear, reads parameters, applies guards, calls `AllocateShearPlanes`
`MGGravityDriver::Solve` `mg_gravity.cpp`	Freezes `mg_qomt_` once per solve
`MultigridDriver::PhysicalBoundary` `multigrid_tasks.cpp`	Calls `FillShearGhostPack` after every boundary exchange
`MultigridDriver::MGRootBoundary` `multigrid_driver.cpp`	Calls `RootShearBoundaryX1` before the $x_1/x_2/x_3$ passes
`MultigridDriver::InitializeOctets` `multigrid_driver.cpp`	`shear_periodic` wraps like periodic in the neighbor search
`ApplyPhysicalBoundariesOctet` `multigrid_driver.cpp`	`shear_periodic` is a wrap, not a wall: no reflection applied

The ordering rule at every one of these sites is the same: *the shear fill runs after the ordinary (unsheared) exchange and overwrites its result.* The exchange performs the plain periodic copy through the neighbor tables; the shear fill replaces that copy with the y-remapped image over the whole slab. This is why the fill is placed at the top of `PhysicalBoundary`, which is the point every boundary exchange funnels through — including the final finest-level exchange that feeds `SelfGravity`'s ϕ ghosts.

4.7 Two corrections uncovered by the plumbing

The `mg_zerofixed` fall-through. Before this work, `shear_periodic` was not a recognized multigrid BC and fell through to `mg_zerofixed`. The consequence was severe and silent: the communication's (correct, unsheared) periodic copy was overwritten with a $\Phi = 0$ Dirichlet wall, and mean-source subtraction was disabled because the domain no longer looked fully periodic. Shearing-box multigrid gravity ran silently wrong. Recognizing `shear_periodic` throughout `mg_mesh_bcs_` fixes both.

Mean-source subtraction. A fully periodic Poisson problem is solvable only if the source has zero mean. `shear_periodic` is a periodic identification, not a physical boundary, so it must count toward "fully wrapped":

```
bool wrap_all = true;
for (int f = 0; f < 6; ++f) {
  BoundaryFlag mbc = pmy_mesh_->mesh_bcs[f];
  wrap_all = wrap_all && (mbc == BoundaryFlag::periodic ||
                          mbc == BoundaryFlag::shear_periodic);
}
if (!wrap_all) { fsubtract_average_ = false; }
```

4.8 Driver-independent task-list execution

Multigrid task lists were run through `Driver::ExecuteTaskList`, which dereferences the `Driver*`. Problem generators are constructed *before* the `Driver` exists, so a static Poisson solve from a pgen (`pgrav->Solve(nullptr, 1)`) crashed with `solver = multigrid`. `MultigridDriver::ExecuteMGTaskList` replicates the runner but tolerates `pdriver == nullptr`,

which is safe because every multigrid task ignores its `Driver*` argument. Together with a null guard in the non-convergence branch of `SolveIterative`, this makes static solves through the full multigrid path a supported configuration — the mechanism behind the static Poisson tests.

4.9 The MPI seam

Multi-rank support required exactly one collective, at one place (Fig. 2). Each rank gathers only its own boundary blocks' columns into a zero-filled plane; one `MPI_Allreduce(MPI_IN_PLACE, ..., MPI_SUM)` then reconstructs the global plane on every rank; remap and scatter proceed rank-locally (redundantly, but the planes are small).

Two properties make this exact rather than merely adequate. First, every plane cell has *exactly one* writer — boundary blocks tile the level's y–z extent — so the summed contributions are one value plus zeros, and the result is independent of reduction order: the multi-rank solve is bit-identical to the serial one. Second, all ranks reach the collective in lockstep, because every early-return at the call site (`shear_plane_ == nullptr`, `ncells < 1`, `mg_qomt_ == 0`) is a level-global condition, not a rank-local one. The `Kokkos::fence()` before the call is required because the gather kernel is asynchronous.

The root grid needs no communication (it is replicated), and the octet fill of Phase 2b needs none either (octets are replicated). The seam is compiled out entirely in serial builds.

The same commit made the `fft_poisson` pgen's diagnostics MPI-aware: the density mean and all residual/error norms were rank-local reductions written under the FFT solver's single-rank assumption, which corrupted the printed RHS mean and scaled the L_2 norms by $1/\sqrt{N_{\mathrm{ranks}}}$ under MPI.

4.10 Phase-1 guards and parameters

Condition	Reason
x_2 or x_3 not periodic	The shear identification assumes a periodic azimuth and vertical; the plane wrap would be wrong
`<gravity> mg_bc` set	An explicit MG boundary override cannot be combined with the shear identification
`root_on_host = true`	The root shear fill is a device kernel
`mg_nghost > 1`	Untested; also the octet fill of Phase 2b assumes `ngh` $= 1$ (octet interiors are 2 cells)
`mg_remap` unrecognized	Must be `dc`, `plm` (default), or `ppmx`

5 Phase 2: multigrid gravity on refined shearing-box meshes

Phase 2 is decomposed into three steps chosen so that each one isolates a single new mechanism, and each can be validated against the previous step as a regression.

Step	Mesh configuration	New mechanism
2a	Interior rings; boundary at root level	none (guard relaxation)
2b	Boundary annuli; both faces refined uniformly	sheared *octet* ghosts
2c	Adaptive refinement	rebuild-on-remesh

5.1 Phase 2a: interior rings

Phase 2a required *no changes to the fill at all*, and understanding why is the point of the step.

The shear fill iterates over blocks and gates on `mb_bcs`. Interior fine blocks do not carry `shear_periodic` on any x_1 face, so the gather and scatter skip them automatically. Every *participating* block is at the root level, so the plane geometry — sized in root-block units at construction — remains correct. The refined interior is handled entirely by the pre-existing octet machinery, which is level-agnostic and does not touch the shear faces.

What changed was the guard. The Phase-1 blanket `multilevel` fatal became a scan requiring every block on an x_1 face to be at the root level, with AMR still fatal (deferred to 2c) and refined boundary faces still fatal (deferred to 2b).

5.2 Phase 2b: sheared octet ghosts

Boundary annuli — both x_1 faces uniformly refined over their full x_2/x_3 extent, allowed by I1 since Phase 0b — put multigrid *octets* on the shear-periodic faces. Those octets exchange x_1 ghosts through the octet neighbor table, which wraps periodically (`InitializeOctets`, patched in Phase 1) but *unsheared*. They need the same treatment as the blocks.

Why this is a separate implementation. The octet regime differs from the block regime in every respect that matters to the fill: octet buffers live in *host* memory, are *replicated on every rank*, and have a fixed 2^3 interior. So `FillShearOctetGhosts` is the Phase-1 algorithm — gather, remap, scatter, same sign convention — re-expressed as serial host code over a `std::vector` plane, with no MPI seam at all.

The scalar remap twins. The conservative remap kernels are Kokkos *team* kernels and cannot be called from serial host code. Rather than duplicate the reconstruction logic, `remap_fluxes.hpp` gains scalar per-face twins — `DC_RemapFlxFace`, `PLM_RemapFlxFace`, `PPMX_RemapFlxFace` — each returning the remap flux at one face of a padded row, with the same convention as the team versions (faces $jf \in [\texttt{PAD}, \texttt{PAD}+\texttt{gny}]$). The upwind cell is $c = jf - 1$ for $\epsilon > 0$ and $c = jf$ for $\epsilon < 0$, matching which face the team kernels write. The device path is untouched, which is what preserves the Phase-1 and Phase-2a results bit-for-bit.

The header carries an explicit maintenance rule: *the math must stay in lockstep with the team versions; any change to one is a change to both.*

Gating. `oct_shear_face_` is a per-octet-level flag set in `InitializeOctets`: level l is flagged when its octets reach *both* $l_{x1} = 0$ and $l_{x1} = (\texttt{nrbx1_} \ll l) - 1$. Under I1 either both faces are tiled at a level or neither is, so a single boolean per level is sufficient.

Hook points. Every consumer of octet ghosts funnels through two chokepoints, and the fill is placed at both, after the per-octet exchange:

- `SetBoundariesOctets(fprolong, folddata)` — per level, during the cycle: `FillShearOctetGhosts(lev, folddata)`;
- `SetOctetBoundariesBeforeTransfer(folddata)` — all levels, before the root transfer, since these ghosts feed `SetFromRootGrid` for the boundary blocks' coarsest levels.

The `folddata` argument propagates to the fill: when set, `Uold` is gathered, remapped, and scattered alongside `U` in the same pass (the plane carries a leading `nfield` axis of size 1 or 2).

Plane sizing at the boundary level. With boundary annuli the participating blocks sit at a uniform level *above* root, so the block planes must span the y–z extent at *that* level. `AllocateShearPlanes` determines it by scanning for the first block on an x_1 face — under I1, any face block gives the answer — and stores `shear_reflev_` $= \ell - \ell_{\mathrm{root}}$, which then multiplies `gny` and `gnz`.

Guard relaxation. The 2a root-level requirement weakens to exactly the hydro invariant I1: the x_1-face blocks must share one uniform level, reported with the observed level span on violation. Interior rings (boundary at root) and boundary annuli (boundary above root) both pass through the same check.

A note on coarse-level fidelity. As in Phase 1, the V-cycle's fixed point is set by the finest block level, so octet-level ghost quality affects only the convergence rate. The fill is still done consistently at every octet level, because an inconsistent coarse-grid correction is what would degrade the sheared contraction factor away from its periodic value.

5.3 Phase 2c: adaptive refinement

AMR invalidates every shear structure that was sized at construction: plane extents (if the boundary level changes), the per-block offset table (load balancing can reassign blocks without changing their count), and the octet face flags.

The fix rides machinery that already exists. `MultigridDriver::PrepareForAMR` is called at the top of every `Solve` and already detects any remesh via an update counter, rebuilding the multigrid arrays and ϕ. Two changes complete the picture:

1. `Multigrid::AllocateShearPlanes` is made *idempotent*: the plane array is allocated only if null, `Kokkos::realloc` resizes each plane if the boundary level changed, `shear_reflev_` is re-derived, and the offset table is recomputed from the current logical locations. It is then called from the same rebuild branch of `PrepareForAMR`.
2. The octet face flags need no new code: `InitializeOctets` is already part of the rebuild and reassigns `oct_shear_face_` from the new octet set.

The constructor's `adaptive` fatal is removed. The invariant it was protecting is not abandoned but relocated: `Mesh::CheckShearingBoxRefinement` re-enforces I1 and I2 after *every* AMR update (§3.2), so by the time `PrepareForAMR` runs, the mesh is known to satisfy the uniform-boundary-level condition that the plane sizing assumes.

Finally, the swing problem generator gains the moving-ring criterion of §3.9, so that AMR + shear + multigrid gravity can be exercised on a problem with a known analytic reference.

5.4 Phase 2d: transverse slopes of the fine ghosts at level jumps

A correction to the general multigrid solver (any refined mesh, shearing box or not), made in November 2026 after R. Li's solver was seen to converge at a level jump where ours did not. Commits `d86c16ac` and `57a70d8d` on branch `fargo-amr-mhd`; the fix is cherry-picked to `dust-multigrid` as `e1e0a072` (the test problem is not). Validation: `multigrid_sb_validation.pdf`, §"The fine-ghost fix".

Where the fine ghosts come from. Between V-cycle smoothing passes and after the final pass, `MultigridBoundaryValues::ProlongateFCMG` (`src/multigrid/multigrid_bvals.cpp`) is the only fill of a fine block's ghost cells on a face whose neighbor is coarser. The coarse values arrive in `coarse_buf_` (`cbuf`) through the `icoar` receive, laid out as the coarse patch facing the block (indices `ngh`$\ldots$`ngh` + `half` -1 in each transverse direction, `half` = `ncells`/2) *plus* `ngh` cells on one side: the receive extents grow by `ngh` toward the parent's interior (`bje += ngh` for parity $f = 0$, `bjs -= ngh` for $f = 1$), since that is where the coarse neighbor continues. A coarser neighbor is stored in the slot `NeighborIndex`$(o_1, o_2, o_3, f_1, f_2)$ with (f_1, f_2) the block's own child parity in the two transverse directions (`mesh/meshblock.cpp`), so the loop variables f_1, f_2 of the face branch are those parities. For an x_1 face, with c the coarse cell across it and u_f the first fine cell,

```
u(fk+a, fj+b, fig) = (2*(c + (2b-1)*gy + (2a-1)*gz) + u(fk+a, fj+b, fi)) / 3
```

for the four fine ghosts (a, b) facing c; the x_2 and x_3 branches are the same with the directions permuted. g_y, g_z are the transverse offsets of the fine ghost centres from the coarse centre, a quarter of a coarse cell times the slope.

The defect. The slope was formed as

```
int sjm = (sj > ngh) ? sj-1 : sj;   int sjp = (sj < ngh+half-1) ? sj+1 : sj;
Real gy = 0.125*(cbuf(..,sjp,..) - cbuf(..,sjm,..));
```

clamping to a one-sided difference at both ends of the patch while keeping the centred weight, so the offset was halved at every clamped cell. With 4-cell blocks (`half` = 2) that is every ghost at the jump; with 16-cell blocks the patch edges. The error, $h_f\,\partial\Phi/6$ in the ghost, is $O(h)$, an $O(1)$ error in the centred force difference; the operator stays conservative because the $\pm$ offsets cancel in the face flux.

The change. In each of the three face branches:

```
int sjm = (sj > ngh || f1 == 1) ? sj-1 : sj;       // halo below when parity 1
int sjp = (sj < ngh+half-1 || f1 == 0) ? sj+1 : sj; // halo above when parity 0
Real gy = 0.25*(cbuf(..,sjp,..) - cbuf(..,sjm,..)) / (sjp - sjm);
```

(and the same with f_2 for the second transverse direction; for an x_2 face the transverse directions are x_1 with f_1 and x_3 with f_2, for an x_3 face x_1 with f_1 and x_2 with f_2). The slope is centred wherever the halo exists and one-sided with the one-cell weight $\frac{1}{4}$ only at the outer edge. Nothing else changes: the normal interpolation, the coarse-side ghost $\frac{1}{3}(4\bar{u} - u)$, octets (`ProlongateOctetBoundariesFluxCons`, which already used full centred slopes), edge and corner injection, buffer sizes and messages. Uniform meshes do not reach this code (`nreflevel` = 0). The unused `FillFineCoarseMGGhosts` still has the old clamp.

Effect. On a static level jump the potential becomes second order and the force first order at the interface (from a non-converging $\sim 10\%$ of the peak force to 0.43% at a 256^2 root), and the refined mesh is no longer less accurate than its uniform root grid. Second order in the force needs a quadratic normal interpolation with a flux-matched coarse ghost (§11).

Test problem. `pgen_name = mg_interface_wave` (`src/pgen/tests/mg_interface_wave.cpp`): R. Li's `self_gravity_wave` Poisson mode for our solver — a cell-averaged single mode, one solve in the problem generator, and the errors of Φ and of the centred-difference force against the exact solution, globally and in the interface cells, printed as one `# MG_INTERFACE_WAVE {...}` line. Parameters `rho0`, `amp`, `phase`, `nwx`, `nwy`, `nwz`; a 2-D problem is run as a 3-D slab one block thick.

5.5 Phase 2e: quadratic level-jump ghosts (`mg_fc_interp`)

After Phase 2d the potential is second order at a level jump but the force is first order, because both ghosts are linear interpolations normal to the jump. The option `<gravity> mg_fc_interp` selects the pair: `linear` (default; the code of §5.4, bit-identical), `quadratic` (quadratic pair on the finest multigrid level, linear elsewhere), or `quadratic_all_levels` (diagnostic: quadratic on every block level with ≥ 4 cells). Read in the `MGGravityDriver` constructor (`src/gravity/mg_gravity.cpp`) and stored with `Multigrid::SetFCInterp`; `Multigrid::FCQuadratic`(shift, ncells) decides per level (shift $= 0$ and ≥ 4 cells per block for `quadratic`). Requires `mg_nghost = 1` (fatal otherwise); an unknown value is fatal. Validation: `multigrid_sb_validation.pdf`, §"Second order: quadratic level-jump ghosts"; derivation and every table: `validation/mg_fc_quadratic_notes.md` on `fargo-amr-mhd`.

Fine side (`ProlongateFCMG`, face neighbor coarser). Normal to the face, with fine cells f_1, f_2 at $h/2$, $3h/2$ and the coarse cell at $-h$, the ghost at $-h/2$ is

```
u_g = (8/15)*ct + (2/3)*u_f1 - (1/5)*u_f2
```

where `ct` is the coarse layer at the ghost's transverse offset $(\xi, \eta) = (\pm\frac{1}{4}, \pm\frac{1}{4})$ coarse cells from a total-degree-2 fit on 3×3 coarse cells of `cbuf`: $c + \xi D_a c + \eta D_b c + \frac{1}{2}\xi^2\delta_a^2 + \frac{1}{2}\eta^2\delta_b^2 + \xi\eta\, m$ (3-point slopes, $\delta^2 = w_0 - 2w_1 + w_2$, mixed term $m = D_a D_b c$). The 3-cell stencils are centred except at the outer edge of the received data (the coarse neighbor's transverse block edge, chosen by the child parity f_1, f_2 as in §5.4), where they are one-sided (`FCQuadStencilStart`); the transverse corner cells are inside the received box. f_2 is the block's own second interior cell.

Coarse side (`ProlongateFCMG`, face neighbor finer, and `FillCoarseMG`). Flux matching with the fine ghosts above gives

```
u_gc = -(1/15)*u_c1 + B - (1/30)*(d2_a + d2_b),   B = (2/3)*A1 + (2/5)*A2
```

($A_{1,2}$: 2×2 averages of the fine block's first and second layers; δ^2 of the coarse block's own first interior layer, centred, one-sided three cells inward at its transverse block edges — the same stencil the fine side used for that cell, so the flux match is exact, `FCQuadCoarseAtGhosts`). In quadratic mode `FillCoarseMG` packs B into the face slot of `coarse_buf_` that carried A_1, so the `icoar` send, its extents, the buffer sizes, the rank-packed metadata, the MPI messages and the same-rank path are all unchanged.

Levels. The finest block level determines the converged answer; the coarser block levels, the octets (`ProlongateOctetBoundariesFluxCons`) and the root grid only drive convergence. Using the nonsymmetric quadratic pair on the finest level only keeps the V-cycle's contraction equal to the linear operator's; on every level it alternates (up to 0.45 per cycle). Levels with fewer than 4 cells per block use the linear pair on both sides (both sides test the level's cell count, so they always agree). Edge and corner neighbors (injection) are unchanged; the 7-point operator reads face ghosts only. The shear-periodic x_1 faces (one level) and the slab x_3 faces (root level) never meet a level jump, and neither side reads its own ghosts transversally, so neither boundary interacts with the new stencils.

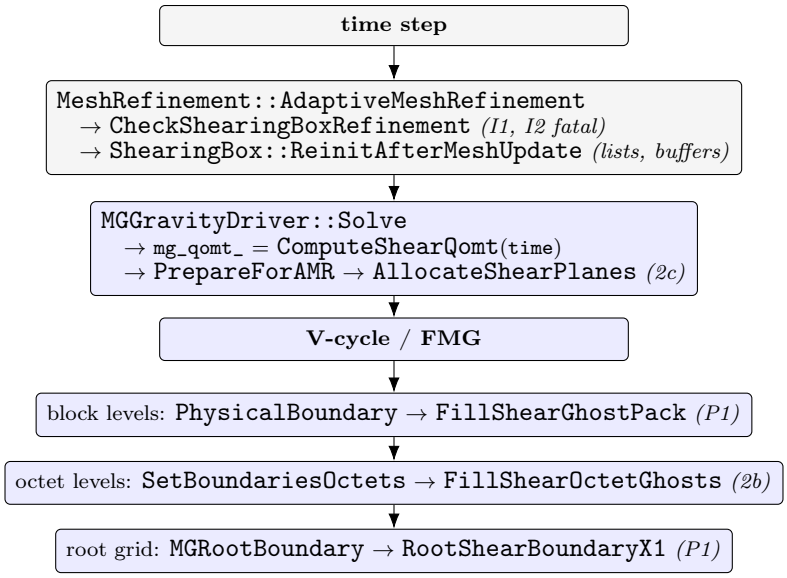


Figure 3: Where each piece runs. Grey: mesh/hydro side (Phase 0). Blue: multigrid solver (Phases 1–2).

5.6 End-to-end control flow

6 Phase 3: the stratified box — vertical gravity, slab-open multigrid boundaries, and cooling

Phases 0–2 built refinement and multigrid self-gravity for the *unstratified* shearing box: gravity was solved on a fully (shear-)periodic domain, which is why mean subtraction (the Jeans swindle) was mandatory and why every boundary of the Poisson problem was an identification rather than a physical edge. Phase 3 adds the third dimension of the physical problem: vertical stratification. The target application is the gravito-turbulent disk of Shi & Chiang (2014, ApJ 789, 34; hereafter SC14) — a 3D, stratified, self-gravitating, cooling shearing box — which requires four new ingredients: (i) the work term of the vertical tidal force in the energy equation, (ii) an *open (vacuum) vertical boundary condition* for the multigrid Poisson solver on a horizontally (shear-)periodic box, (iii) optically thin cooling source terms, and (iv) a problem generator with SC14's self-gravitating hydrostatic initial state and diagnostics.

6.1 Phase 3a: hydro-side stratification terms

The missing energy work term. The momentum-side vertical tidal force $-\rho\Omega_0^2 z\,\hat{z}$ already existed behind `<shearing_box> stratified = true`, but the corresponding work term in the energy equation — $-\rho\Omega_0^2 z\,v_z$, cf. SC14 eq. (3) — was absent in all four source-term branches (hydro/MHD $\times$ FARGO/non-FARGO). For isothermal runs (the only stratified configurations exercised upstream) this is invisible; for an ideal-gas stratified box it is a first-order energy error. All four branches now apply

$$\Delta E = -\,\Delta t\,\rho\,\Omega_0^2 z\,v_z, \tag{12}$$

consistent with the existing convention of building source terms from primitives.

Cooling. Two SC14 prescriptions were added to `SourceTerms` (`srcterms/srcterms.cpp`), both operating on the internal energy density U:

$$\texttt{beta_cooling}: \qquad \dot{U} = -U\,\Omega_0/\beta, \qquad \beta = \texttt{bcool_beta}\ (= \Omega_0 t_{\rm cool}), \tag{13}$$

$$\texttt{thermal_cooling}: \qquad \dot{U} = -U/t_{\rm cool}, \qquad t_{\rm cool} = b\,(\rho/P)^3 \quad (\texttt{tcool_b}), \tag{14}$$

the second being SC14's optically thin constant-opacity law (their eq. 8). Ω_0 defaults to `<shearing_box> omega0`. `SourceTerms::NewTimeStep` caps the step at `cool_eps` $t_{\rm cool}$ (default 0.02, SC14's choice); for `beta_cooling` the cap is a constant and never binds in practice.

Vertical hydro boundaries: diode, not outflow. SC14 describe their vertical boundaries as "outflow." With AthenaK's plain `outflow` BC (ghost = copy of the last active cell) the stratified box is *unstable to an inflow runaway*: once gravity establishes an inward velocity at the z faces, the copied ghost state feeds mass at that velocity indefinitely. In the hydrostatic-hold test below, the halo density grew by two orders of magnitude in $10\,\Omega_0^{-1}$ and the accreting fountain destroyed the disk. The `diode` BC (outflow that zeroes inbound normal velocity in the ghosts) removes the reservoir; the reference inputs use it on both z faces.

6.2 Phase 3b: slab-open vertical boundaries for the multigrid solver

Problem statement. The Poisson problem of the stratified box is (shear-)periodic in x_1, periodic in x_2, and *open* in x_3: the correct exterior solution is vacuum decay, mode by horizontal mode. The FFT solver already provides this configuration via the Koyama & Ostriker (2009) two-solve method (`vert_bc = open`), but only on uniform single-rank grids; making the *multigrid* solver support it is what buys refinement compatibility.

Method: exact Dirichlet face planes from the discrete vacuum Green's function. For each horizontal Fourier mode of the rolled (strictly periodic) frame, with discrete horizontal eigenvalue

$$\lambda_\perp = \frac{2\cos(k_x'\Delta x_1) - 2}{\Delta x_1^2} + \frac{2\cos(k_y \Delta x_2) - 2}{\Delta x_2^2} \le 0, \qquad k_x' = k_x + q\Omega_0 t_s \tfrac{L_x}{L_y} k_y, \tag{15}$$

the 7-point stencil reduces in z to a three-term recurrence whose decaying solution is $\mu^{|\Delta n|}$ with

$$\alpha = 2 - \lambda_\perp \Delta x_3^2 \ (\ge 2), \qquad \mu = \frac{2}{\alpha + \sqrt{\alpha^2 - 4}} \in (0, 1]. \tag{16}$$

The infinite-vacuum discrete Green's function is

$$G(\Delta n) = -\frac{\Delta x_3^2\,\mu^{|\Delta n|}}{\sqrt{\alpha^2 - 4}}, \qquad \text{and for } k_\perp = 0: \quad G(\Delta n) = +\frac{\Delta x_3^2\,|\Delta n|}{2}, \tag{17}$$

the latter being the discrete isolated-slab potential $2\pi G\Sigma|z|$ in the symmetric gauge. Each `Solve()` computes the *face-value* planes

$$\hat{\Phi}_{\rm face}(k_\perp) = \sum_{n'} \hat{S}(k_\perp, n')\,\tfrac{1}{2}\left[G(n_{\rm last} - n') + G(n_{\rm ghost} - n')\right], \qquad \hat{S} = \widehat{4\pi G\rho}, \tag{18}$$

on both x_3 faces and imposes them multipole-style at every multigrid level:

$$u_{\rm ghost} = 2\,\Phi_{\rm face} - u_{\rm interior}. \tag{19}$$

Because the face value is the mean of the infinite-domain solution at the last active and first ghost centers, the converged interior solution is *exactly* the restriction of the infinite-vacuum discrete solution — there is no additional boundary-truncation error, and (for $k_\perp \neq 0$, and for the mass sector in the symmetric gauge) no free constant. The FAS fixed point is unaffected by the coarse-level plane restrictions: at convergence the coarse FAS right-hand side and smoother see the same imposed ghosts, so the coarse correction vanishes identically.

Pipeline. `MultigridDriver::ComputeSlabPlanes` (`multigrid/multigrid_slab.cpp`), once per solve:

1. gather $4\pi G\rho$ at root resolution — refined blocks are conservatively averaged into the root sampling ($2^{3\ell}$-cell means via atomics); each rank fills only its blocks and one `MPI_Allreduce(SUM)` reconstructs the global array on every rank (the Phase-1 zero-fill idiom, so every rank then computes identical planes with no further communication);
2. roll into the periodic frame with the conservative remap kernels ($+q\Omega_0 t_s x$ per column, in place — each team stages its full column in scratch);
3. per-mode μ and face weight; then slice-by-slice 2D FFTs (kokkos-fft, reusable plans) accumulating both face spectra with μ^n read in ascending order (the outer face reads slices reversed, so no small-μ division);
4. inverse-transform both planes, unroll, and write into the interior of pyramid level 0;
5. build the *plane pyramid*: one plane pair per global multigrid level (pyramid index p = block level shift, then $p = n_{\mathrm{mblevel}} - 1 + \ell_{\mathrm{root}}$), each carrying `ngh`-deep ghost rings — periodic wrap in x_2, shear-remapped wrap in x_1 — restricted by 2×2 face averages.

Corner consistency. The x_1 shear fill wraps its z-edge corners periodically, which is wrong for a slab face; the fix is ordering, not surgery. In both the block-level `PhysicalBoundary` task and `MGRootBoundary`, the x_3 pass runs *last* over the full slab including x_1/x_2 ghost columns, reading the plane's ghost rings — so the slab extrapolation overwrites the wrapped corners with values consistent with the neighboring interior ghosts.

Interaction with refinement. The plane pyramid indexes block levels by their shift relative to the *root* resolution, which requires every block touching an x_3 face to be at root level (`CheckSlabBlockLevels`, re-enforced after every AMR remesh in `PrepareForAMR`); consequently no x_3-boundary octets exist, and a defensive fatal guards `ApplyPhysicalBoundariesOctet`. Physically this is no restriction for the target problems: refinement chases midplane structure, and the middle z-row of 16^3 blocks in the SC14 geometry spans exactly $|z| < 2H$.

Mean subtraction and gauge. With a physical x_3 boundary the domain is no longer fully periodic, so the existing `wrap_all` logic disables mean-source subtraction automatically; the Dirichlet planes fix the constant. The mass sector is now *physical* (no Jeans swindle) — the horizontally averaged potential is the discrete slab potential, a sector no periodic-box test could ever probe.

Addendum (2026-09-08): a distributed plane computation. The first implementation of `ComputeSlabPlanes` gathered the whole root-resolution $4\pi G\rho$ into a (n_z, n_y, n_x) array on every rank (an Allreduce of the full box), rolled it into the periodic frame with the conservative remap, and transformed all n_z planes on every rank. Its cost therefore grew with the *box*, not with a rank's share of it: at $512^2 \times 256$ on 2048 ranks that was 0.54 GB per rank per solve and ~ 25 s per

cycle — the dust track's Baehr et al. box could not run. The rewrite keeps the method and changes the bookkeeping. Each rank builds, for each root k-plane its own blocks touch, a padded (n_y, n_x) plane holding only its cells (refined blocks averaged conservatively as before); transforms it along y; applies the roll $\rho'(x, y) = \rho(x, y - q\Omega t\, x)$ as the exact phase $e^{-ik_y q\Omega t\, x}$ per column; transforms along x; and accumulates its contribution $\hat{\rho}_k\, w_{\mathbf{k}}\, \mu^k_{\mathbf{k}}$ (and its mirror) to the two face spectra. Because the transform, the phase and the accumulation are all linear in the density, one Allreduce of the two (n_y, n_x) complex planes (8 MB) yields exactly the spectra of the whole box. The inverse is the mirror image (inverse x transform, conjugate phase, inverse y transform) and lands directly in level 0 of the pyramid; the ring fill and restriction (step 7) keep the remap, since the rings must match the multigrid's own ghost convention. Two consequences beyond speed: the density roll is now free of remap error (the sheared gravito-turbulence history matches its archive to every digit until the shear phase builds and differs at 10^{-6}–10^{-4} after), and the result is rank-independent — the limited remap is nonlinear, so with it the sum of remapped partial planes was not the remap of the sum, and 4 ranks differed from serial by 10^{-3} in the sheared sheet test; now they agree to 10^{-7}. Static tests: the sheet error 1.203474×10^{-12} is unchanged to every digit, serially and on 4 ranks. Cost: a 64^3 stratified box went from 25 s to 0.33 s per cycle on one core; 10^6 cells run at 1.1 s per cycle. The "memory-bandwidth-bound" per-cycle costs quoted for the cluster runs were dominated by the old gather and no longer apply.

6.3 Phase 3 validation

Sheet test (new `profile = sheet` in the `fft_poisson` pgen). A uniform background ρ_0 plus one horizontal mode on a single z-layer has a *closed-form* infinite-vacuum discrete solution (μ-decay for the mode, the discrete parabola for the background), compared *without mean subtraction*: an absolute test of the boundary machinery, constant included. Measured (`tests/mg_poisson_slab.athinput`, 64^3, 16^3 blocks):

Configuration	max rel. error	notes
uniform, $q\Omega_0 t_s = 0$	1.2×10^{-12}	solver tolerance; BCs exact
uniform, MG vs FFT-KO09	3.0×10^{-13}	independent-method agreement
sheared ($t_0 = 0.37$), MG	6.6×10^{-6}	PPMX roll/unroll remap order
sheared, MG vs FFT	8.8×10^{-8}	both roll, different pipeline points
4 MPI ranks (all three)	bit-identical	to the serial digits above

Physics test. The horizontally uniform sech^2 slab against the analytic $4\pi G\rho_0 H^2 \ln\cosh(z/H)$ potential converges at second order (5.8×10^{-3} / 1.5×10^{-3} / 3.7×10^{-4} at $32/64/128^3$), and the $n = 64$ MG-slab and FFT-open results agree to every printed digit.

Refined meshes (`tests/mg_poisson_slab_smr.athinput`). With a level-1 slab spanning the middle half of z: the screened mode sector converges at second order (1.6×10^{-2} / 4.6×10^{-3} / 1.2×10^{-3} relative). The unscreened $k_\perp = 0$ sector exhibits a *constant, horizontally uniform* offset of exactly $\frac{3}{4}\,(4\pi G\rho_0)\, h_f^2$ filling the refined region — the $O(h^2)$ dipole signature of the Tomida & Stone conservative level interface (zero monopole: the flux error is continuous across the interface, so no fake mass; the dipole layer produces a potential jump). Being constant it exerts *zero force*. Diagnosing this cost a detour: the sheet metric initially sampled its reference at integer root-layer indices, which on fine cells adds an $O(\partial_z\Phi\, \Delta z)$ *reference* error that masqueraded as a first-order solver error; the metric now evaluates the closed form at the true fine cell centers.

Evolution twin (MG-slab vs FFT-KO09). The same small $\beta = 10$ gravito-turbulence box evolved through both gravity solvers to $t = 5\,\Omega_0^{-1}$: after removing the constant gauge offset between the two open-boundary conventions ($\simeq 2\pi G\Sigma$-scale, 25.6 here), the potentials agree to 8×10^{-5} **rms** and density/thermal energy to 10^{-3}, while the small random-seeded perturbation velocities decorrelate chaotically (relative difference $0.3 \to 0.9$) — the signature of two correct solvers separated only by roundoff-level method differences, with gravity demonstrably *not* the driver of the separation.

Adaptive refinement under slab gravity (`shearing_box/gravito_turb_amr_test.athinput`). A $16H$-wide $\beta = 10$ box with a density criterion (refine at $\rho > 2\rho_0$) run through the cooling collapse: refinement fired on the collapse clumps at $t \sim 15$ ($\rho_{\max} = 38$ at $t = 20$), tracked them, and derefined as they sheared apart — multiple remesh events in both directions with the slab planes rebuilt each time, zero guard trips, and fine blocks confined to the middle z-row ($|z| \le 2H$) throughout by the automatic slab-z refinement policy. (A first attempt exposed a policy interaction worth knowing: a box with only two x_1 MeshBlock columns cannot refine *anywhere*, because both columns sit on the shear-periodic faces that the Phase-2 policy protects; the test input therefore provides four columns.)

Hydrostatic hold. The SC14 initial state (Q3c below) run without cooling or perturbations holds for $10\,\Omega_0^{-1}$: midplane density unchanged, disk-body velocities $\lesssim 2\%$ of $c_s(0)$ (the settling of the non-equilibrium 10^{-4} halo splashing on the disk surface), total mass constant to 0.25% (density-floor injection in the halo). With plain `outflow` instead of `diode` the inflow runaway destroys the disk by $t \sim 8\,\Omega_0^{-1}$ — the origin of the diode default.

6.4 Phase 3c: the SC14 problem generator

`ProblemGenerator::GravitoTurb` (`pgen/fluids/gravito_turb.cpp`, `pgen_name = gravito_turb`) integrates the vertical hydrostatic balance (SC14 eq. 9)

$$\frac{1}{\rho}\frac{dP}{dz} = -\,\Omega_0^2 z - 4\pi G \int_0^z \rho\, dz', \qquad P = K\rho^\gamma, \quad K = c_s(0)^2/(\gamma\rho_0^{\gamma-1}), \tag{20}$$

by midpoint rule on a 2^{15}-point table, floors the profile at `dfloor_ic` (10^{-4}), and interpolates to cell centers. With SC14's published constants ($c_s(0) = 2.12625$, $4\pi G = 4.25231$, $\gamma = 5/3$, $\rho_0 = \Omega_0 = 1$) the computed half-thickness $\int_0^{z_{\max}} \rho\, dz/\rho_0 = 1.0005$, confirming their $Q_0 = 1$, $H = 1$ calibration independently. Cell-wise random velocity perturbations (up to `pert_amp` $c_s(0)$ for $|z| <$ `pert_zmax`) use a splitmix64 hash of *global* cell indices: decomposition- and rank-independent. The user history emits the volume integrals behind SC14's diagnostics — mass, $\int \rho c_s$, the density- and volume-weighted stresses $g_x g_y/4\pi G$ and $\rho v_x \delta v_y$, $\int \rho P$, $\int \rho c_s^2$, $\int \rho\, \delta v^2$, thermal energy — from which Q (eq. 16), α (eq. 19), α' (eq. 20) and rms δv are formed in post-processing. A short small-box run reproduces SC14's transient phenomenology: cooling contraction ($Q : 0.9 \to 0.6$), gravitational instability onset, rebound toward $Q \sim 1$ with $\delta v \sim c_s$.

7 Phase 4: ideal MHD with self-gravity

Phase 4 couples a magnetic field to the self-gravitating shearing box: first as a gravito-turbulent disk threaded by a field (the eventual target: collapse in the outer disk with ambipolar diffusion), and first of all on test problems with an analytic answer. Two pieces of code carry it. The SC14 problem generator `pgen/fluids/gravito_turb.cpp` now runs with an `<mhd>` block as well as `<hydro>` (commit `2a0287f3`), and the Jeans-wave problem generator `pgen/tests/jeans_wave.cpp` gained a

magneto-Jeans mode in which the field is oblique to the wave. Everything the MHD fluid needs from the rest of the code — constrained transport, HLLD, orbital advection of face fields, the multigrid source terms — already existed; the work is in the problem generators, the vertical boundary, and the floors.

7.1 Phase 4a: the gravito-turbulent box with a field

Either fluid. Every access to the gas goes through two helpers, `GasU0` and `GasW0`, which return the `<mhd>` arrays when that fluid exists and the `<hydro>` arrays otherwise; the shearing box and the equation of state are taken from whichever fluid exists. Dust particles with MHD stop with a fatal error (the particle insertion and drag are hydro-only).

Initial field. The hydrostatic profile is unchanged; a field uniform in z does not enter it. `<problem> b_config` selects

`vertical`	$B_z = B_0$: net vertical flux
`vertical_znf`	$B_z = B_0 \sin(2\pi x/L_x)$: zero net flux
`none`	$B = 0$

with $B_0 = \sqrt{2P_0/\beta_p}$ from the midplane plasma beta `plasma_beta` $= \beta_p$ and $P_0 = K\rho_0^\gamma$ (Heaviside–Lorentz units, magnetic pressure $B^2/2$). Only x_3 faces are set, so $\nabla\cdot B = 0$ holds exactly on the faces; the magnetic energy is added to E from the cell-centred field. The MRI wavelength $\lambda = 2\pi v_A/\Omega_0$ makes the resolution requirement explicit: at 4 cells per H a net-flux MRI is resolved only for $\beta_p \lesssim 100$ ($\lambda = 0.46H$ at $\beta_p = 10^3$).

The Alfvén ceiling. The halo is tenuous (ρ down to the floor, $10^{-4}\rho_0$), so field that reaches it drives v_A, and with it the time step, to arbitrary values: a uniform B_z at $\beta_p = 10$ gives $v_A = 74$ at the floor and $\Delta t = 10^{-3}$, ten times below the hydro step. `<problem> va_max` > 0 imposes a ceiling as a density floor,

$$\rho \;\geq\; B^2/v_{A,\max}^2, \tag{21}$$

applied in `GravitoTurbAlfvenFloor` to every active cell after every stage (as a user source term). Where it acts, density, momenta and passive scalars are multiplied by $f = \rho_{\rm floor}/\rho$, and the non-magnetic energy is multiplied by f with the magnetic energy left alone, $E \to (E - B^2/2)f + B^2/2$: velocity and temperature are kept, the field is untouched. The field used is the cell-centred one of the last primitive recovery, because the CT update of the stage comes after the source terms in the task list; the ceiling therefore lags the field by one stage. The mass added is accumulated per rank and written as the history column `dm_vafloor` (the history output sums ranks). The default input sets $v_{A,\max} = 20$: the saturated hydro run's CFL step at 4 cells per H ($\Delta t \approx 3.5 \times 10^{-3}$) corresponds to a signal speed of about 21, so the ceiling holds the MHD step near the hydro one.

Mass renormalization with a field. The Booth & Clarke rescaling (§6) multiplies the conserved variables by $M_{\rm target}/M$ once per step. With MHD the field is not rescaled — the net flux is a conserved parameter of the box — and the energy is rescaled as above, $E \to (E - B^2/2)f + B^2/2$.

History. With MHD five columns follow the hydro (and dust) ones: `rho_wmax` $= \int \rho(-B_xB_y)\,dV$ and `wmax` $= \int -B_xB_y\,dV$ (the Maxwell stress, to be added to `wgrv`+`wrey` in α'), `emag` $= \int B^2/2\,dV$, `babs` $= \int |B|\,dV$, and `divb` $= \int |\nabla\cdot B|\,\Delta x_{\min}\,dV$ with $\nabla\cdot B$ from the face fields; `divb`/`babs` is the mean relative divergence.

The hydrostatic vertical boundary with a field. `hse_outflow` (§6) rebuilds the ghost energy from the edge cell's temperature $a^2 = (\gamma-1)(E-E_k)/\rho$, which with MHD would count the magnetic energy as heat: a halo cell at $\rho = 10^{-4}$ in a field of $\beta_p = 10$ would get a^2 larger by $B^2/2\rho \approx 2700$. It had been made fatal for MHD; it is now supported. The field boundary (`bfield_bcs.cpp`) treats `hse_outflow` as `outflow`, copying the edge faces into the ghosts. Because the CT update and the field exchange precede the physical boundaries in the task list, the ghost's cell-centred field is known in `BCHelperHydro` from the active faces alone: B_x, B_y as in the edge cell, B_z equal to the boundary face. The helper takes the face arrays of the fine or coarse level (a new `coarse` argument) and uses

$$a^2 = (\gamma-1)\,\frac{E - E_k - B^2_{\rm edge}/2}{\rho}, \qquad E_{\rm ghost} = \frac{\rho_g a^2}{\gamma-1} + E_{k,g} + \frac{B^2_{\rm ghost}}{2}. \tag{22}$$

Cooling for the MHD runs. `BetaCooling` reads the internal energy from the primitives and subtracts from u_0's total energy, so it applies to MHD unchanged. Two options added for the fragmentation study apply to both fluids (commit `29246ce6`): `bcool_rho_cut`, which multiplies the cooling rate by $1/(1+(\rho/\rho_{\rm cut})^4)$ (a first-core cutoff), and a linear ramp of β from `bcool_beta0` at $t = 0$ to `bcool_beta` at `bcool_t_ramp`, evaluated at the middle of each step.

Running it. `inputs/shearing_box/gravito_turb_l64z16_mhd.athinput` is the $64H \times 64H \times 16H$ box at 4 cells per H with HLLD, FOFC, net vertical flux, $\beta_p = 10^3$ and $v_{A,\max} = 20$. The NAS runner `scripts/cluster/gt_l64z16.pbs` takes it through `INPUT=` with an MHD build (`BUILD=build-mhd`); it checks that the binary carries the MHD problem generator, and a chained resubmission keeps the input and the node count. MHD runs start cold: there is as yet no way to add a field to a saturated hydro restart.

7.2 Phase 4b: the magneto-Jeans problem generator

The mode. With MHD and `<problem> b_angle` given, `ProblemGenerator::SelfGravity` sets up a uniform field $\mathbf{B}_0 = B_0(\cos\theta\,\hat{\mathbf{k}} + \sin\theta\,\hat{\mathbf{e}}_\perp)$, $\theta =$ `b_angle`, $\hat{\mathbf{e}}_\perp = (-\sin a_3, \cos a_3, 0)$ the unit vector perpendicular to $\hat{\mathbf{k}}$ in the plane of $\mathbf{B}_0$ (a_2, a_3 the wave's angles). Linearising ideal (or isothermal) MHD with self-gravity about this state, a compressive wave with its perturbed field along $\hat{\mathbf{e}}_\perp$ obeys (Chandrasekhar 1961, ch. XIII)

$$\left(\omega^2 - \Omega_J^2\right)\left(\omega^2 - k^2 v_A^2\cos^2\theta\right) = k^2 v_A^2 \sin^2\theta\,\omega^2, \qquad \Omega_J^2 = k^2 c_s^2 - 4\pi G\rho_0, \quad v_A^2 = B_0^2/\rho_0. \tag{23}$$

`mode = slow | fast` selects the smaller or larger root. At $\theta = 0$ and 90° one root carries no density (the transverse $k^2 v_A^2$ wave, and a static mode) and the compressive root is taken whatever `mode` says: $\omega^2 = \Omega_J^2$ and $\omega^2 = k^2(c_s^2 + v_A^2) - 4\pi G\rho_0$ respectively.

The eigenmode. With $\xi = \hat{\mathbf{k}}\cdot\mathbf{x}$ and $\delta\rho = \rho_0 A \sin k\xi$, the perturbed field is $\delta B_\perp = C\,\delta\rho$ with

$$C = \frac{B_\perp\,\omega^2}{\rho_0\,(\omega^2 - k^2 v_{A\parallel}^2)}, \tag{24}$$

and the pressure perturbation is isentropic, $\delta p = c_s^2\delta\rho$. A stable mode ($\omega^2 > 0$) is started with no velocity, which is a standing wave $\delta\rho \propto \cos\omega t$; an unstable one ($\omega^2 = -s^2$) gets $\delta v_\parallel = (s/k)A\cos k\xi$ and $\delta v_\perp = B_\parallel C A (k/s)\cos k\xi$, which is pure growth $\propto e^{st}$. The perturbed field is built from a vector potential $\mathbf{A} = a(\xi)\,\hat{\mathbf{n}}$, $a = -(\delta B/k)\cos k\xi$, $\hat{\mathbf{n}} = \hat{\mathbf{e}}_\perp \times \hat{\mathbf{k}}$ (so that $\nabla\times\mathbf{A} = \delta B\,\hat{\mathbf{e}}_\perp \sin k\xi$), evaluated on the cell edges (`JWVecPot`) and differenced onto the faces, so $\nabla\cdot B$ vanishes to round-off on the grid.

Geometry and diagnostics. `wave_dir = x1` puts $\mathbf{k}$ along x_1 with $\lambda = L_1$, so a thin three-dimensional box is a cheap one-dimensional test; the default (`oblique`) keeps the original box-diagonal wave. The user history (`JeansWaveHistory`, `<problem> user_hist = true`) projects the solution onto the mode, $(2/V)\int(\cdot)\{\sin,\cos\}k\xi\,dV$: `a_sin`, `a_cos` (density), `bp_sin` ($\delta B_\perp$), `vpar_cos`, `vperp_cos` (velocities), then `divb` $= \int|\nabla\cdot B|\,\Delta x_{\min}\,dV$ from the face fields, `babs` $= \int|B|\,dV$ and `nmb`, the number of MeshBlocks. `JeansWaveErrors` uses the stored unstable flag rather than $\lambda/\lambda_J > 1$, since a field across $\mathbf{k}$ can stabilise a hydro-unstable wavelength. Inputs without `b_angle` behave as before (field along x_1). `inputs/tests/magneto_jeans.athinput` is the quasi-one-dimensional setup ($64\times8\times8$ cells, HLLD, PPM4, RK3, multigrid with eight V-cycles after full multigrid), and `validation/run/magneto_jeans/magneto_jeans.py` runs and fits the suite. The gas is adiabatic, $\gamma = 5/3$ and $p_0 = 0.6$, so that $c_s = 1$; the eigenmode is isentropic and eq. (23) holds with $c_s^2 = \gamma p_0/\rho_0$. It was first isothermal, and AthenaK's isothermal HLLD turned out to amplify grid-scale noise at $\beta \lesssim 1$ (§7.4).

Refined meshes. `<problem> amr_mode` selects the refinement criterion: `jeans` (the default; cells per Jeans length) or `window`, enrolled as `JeansWaveWindowRefinement`: every MeshBlock whose x_1 extent overlaps $|x_1 - x_c(t)| <$ `win_hwidth` (periodically) is flagged for refinement and every other one for derefinement, with $x_c(t) =$ `win_x0` $+$ `win_speed`$\,t$ wrapped into the box. A slab of refinement is swept through the wave, so MeshBlocks are created and destroyed while it propagates. `inputs/tests/magneto_jeans_smr.athinput` (a static level-1 slab, $x_1 \in [0.25, 0.5)$) and `magneto_jeans_amr.athinput` (the moving window) are the refined inputs, and `validation/run/magneto_jeans_amr/magneto_jeans_amr.py` runs that suite, adding a nested level-2 region or a 3-D central block by patched copies of the input.

A noise seed. `<problem> noise_amp` > 0 adds a white-noise velocity $\delta v_i =$ `noise_amp`$\,(r - \frac{1}{2})$ to every cell, r a hash of the cell position (identical on any mesh and decomposition). The quasi-one-dimensional runs are otherwise exactly one-dimensional to the last bit, so without a seed no transverse numerical mode can be measured; the seed was added to isolate the isothermal HLLD instability. The 42-case suite (validation document, Phase 4) reproduces eq. (23) to a relative error in ω of at most 4.8×10^{-3} at 64 cells per wavelength and converges at second order.

7.3 Phase 4c: the swing problem generator with a field

`pgen/tests/swing.cpp` (§5.1) now runs with an `<mhd>` block. The field is uniform and azimuthal, $\mathbf{B} =$ `b0`$\,\hat{\mathbf{y}}$, set on the x_2 faces, which keeps $\nabla\cdot B = 0$ exactly, and for an ideal-gas equation of state the density wave carries the isentropic pressure perturbation $\delta p = \gamma p_0\,\delta\rho/\rho_0$ and the field's energy $B_0^2/2$ (the hydro path is unchanged bit for bit: uniform pressure). With $k_z = 0$ and the wave $\delta\rho/\rho_0 = \delta\cos(\mathbf{k}\cdot\mathbf{x})$, $\delta\mathbf{v} = \mathbf{w}\sin(\mathbf{k}\cdot\mathbf{x})$, $\delta\mathbf{B} = \mathbf{c}\cos(\mathbf{k}\cdot\mathbf{x})$, linearised ideal MHD with self-gravity in the shearing sheet gives (Kim & Ostriker 2001 for the thin-disc analogue)

$$\begin{aligned} \delta' &= -(k_x w_x + k_y w_y), & w_x' &= 2\Omega w_y + k_x\Psi\delta - \frac{B_0}{\rho_0}(k_y c_x - k_x c_y), \\ w_y' &= -(2-q)\Omega w_x + k_y\Psi\delta, & c_x' &= k_y B_0 w_x, \qquad c_y' = -k_x B_0 w_x - q\Omega c_x, \end{aligned} \tag{25}$$

with $k_x(t) = k_{x0} + q\Omega k_y t$ and $\Psi = c_s^2 + 4\pi G\rho_0/D$, D the discrete Laplacian eigenvalue. The magnetic tension acts only on w_x (the y component of $(\mathbf{B}_0\cdot\nabla)\delta\mathbf{B} - \nabla(\mathbf{B}_0\cdot\delta\mathbf{B})$ vanishes), and $k_x c_x + k_y c_y = 0$ is conserved. The history gains `bx_cos`, `by_cos` (c_x, c_y), `divb`, `babs` and `nmb`. `inputs/tests/swing_mhd.athinput` sets $B_0 = 0.5$, $\gamma = 5/3$, $p_0 = 0.6$, and `validation/run/swing_mhd/` holds the linear

integrator (`swing_ode.py`) and the runner. Because MHD with orbital advection cannot refine, the refined runs integrate the full shear flow (`orbital_advection = false`), for which the problem generator puts $v_y = -q\Omega x$ in the initial state. An optional uniform radial field `<problem> bx0` is wound up exactly, $B_y =$ `b0` $-q\Omega$ `bx0` t (with `amp = 0`), a kinematic check of the field's shear term.

Reproducer inputs (2026-10-27). `inputs/tests/swing_mhd_repro.athinput` is the minimal case of the refined-mesh field error (validation document, Phase 4): a thin 3-D box ($64 \times 64 \times 4$, $8 \times 8 \times 4$ MeshBlocks), $B_0 = 1$, no gravity, no orbital advection, a static ring $|x| < \pi/4$; it carries `prolong_primitives` and `iso_sound_speed` keys so that both can be switched from the command line, and `validation/run/swing_mhd_repro/repro.py` runs its variants. `inputs/tests/linear_wave_mhd_tflow_smr.athinput` (with `validation/run/tflow_smr/tflow_smr.py`) is the control without a shearing box: a 2-D MHD linear wave across a static slab with a uniform flow and field along the slab faces, set in the wave frame of `pgen/tests/linear_wave.cpp` so that both point along x_2 on the grid.

7.4 Phase 4: what is not there yet

- **MHD with orbital advection on a refined mesh** is still fatal (`mhd/mhd.cpp`; limitation 2 of §11). The gravito-turbulent box with a field can therefore not use AMR until the face-field remap at fine/coarse interfaces is written; problems without a shearing box (the magneto-Jeans wave) can.
- **Seeding a field into a saturated hydro run**: a restart cannot add the `<mhd>` fluid; a converter or a restart-time insertion is needed.
- **Ambipolar diffusion** exists in AthenaK only with a constant coefficient, $\eta_A =$ `eta_ad` B^2 (so $Am \propto 1/\rho$, the wrong trend for collapsing gas), and only explicitly (`sts` refuses it).
- The Jeans-number refinement criterion of `gravito_turb` ignores magnetic support.
- **MHD shearing boxes on refined meshes without orbital advection** carry a 10–20% error in the field perturbation of the magnetized swing test that barely converges with resolution and needs the field, the refinement and the velocity shear together (validation document, Phase 4); gravity and $\nabla \cdot B$ are fine. The fine/coarse EMF correction (`RecvAndUnpackFluxFC`) is exact; the error is already in the fine block's own interface EMFs, which it builds from ghost cells prolongated from the coarse level. Next: the prolongation of face fields and cell-centred variables (`bvals/prolongation.cpp`) in a sheared flow.
- **Conserved-variable prolongation in an adiabatic shear flow.** The ghost-cell total energy carries $\frac{1}{2}\rho v_y^2$, quadratic in x; linear prolongation of the conserved variables gives a wrong ghost pressure, and the background shear flow develops a spurious v_x at the interfaces (hydro and MHD, without orbital advection). `<mesh_refinement> prolong_primitives = true` keeps it exact; it is not on by default.
- **2-D shearing boxes are x–z only.** `shearing_box_r_phi` is not implemented, and a 2-D x–y input runs silently with plain periodic x_1 boundaries; a guard would make it fatal.
- **AthenaK's isothermal HLLD amplifies grid-scale noise in strong fields.** In a uniform medium at rest ($c_s = 1$, $v_A = 1.5$, $\beta = 0.89$, $128 \times 8 \times 8$ cells, no gravity) a 10^{-8} velocity seed grows at $s \approx 15$ in amplitude, with PLM or PPM4 and CFL 0.3 or 0.1; at $\beta = 8$ it does not grow, and the isothermal HLLE and LLF solvers and the adiabatic HLLD ($\gamma = 5/3$, $\beta = 0.53$) are stable. The isothermal branch of `mhd/rsolvers/hlld_mhd.hpp` follows Mignone (2007) on

reading; the cause is not found. The production box is adiabatic; isothermal strong-field runs should use HLLE until it is.

8 Rank-consistent integer shifts

This fix is not part of any phase but is required for all of them at scale, and is recorded here because the failure mode is instructive.

Symptom. A 256^2 run aborted at 64 ranks with Cray MPICH *message truncated*, deterministically at cycle 7900, $t = 13.98274$, as $q\Omega_0 t_s$ approached the shear-periodic wrap — with a 3:4:5 size signature characteristic of a disagreement about how many blocks a message spans.

Cause. The integer cell shift of the x_1-boundary exchange was computed independently on the sending and receiving blocks as

$$\texttt{joffset} = \lfloor y_{\rm shear}/\texttt{mb_size.dx2} \rfloor \,. \tag{26}$$

Per-block `dx2` values carry last-ulp roundoff — they are derived from each block's own extents — so whenever $y_{\rm shear}/\Delta x_2$ sits within an ulp of an integer, the two sides can differ by one cell. That flips the case-1/2/3 selection in the exchange, and hence the `joffset`-dependent MPI *message sizes*, desynchronizing sender and receiver.

Fix. `ShearingBox::ConsistentDx2(gid)` derives Δx_2 from global quantities only, with the exact power-of-two level factor:

$$\Delta x_2(\ell) = \frac{x_{2,\max} - x_{2,\min}}{\left(N^{\rm root}_{{\rm mb},x2}\, n_{x2}\right) \ll (\ell - \ell_{\rm root})}. \tag{27}$$

Every input is bitwise identical on every rank, so every block of a level — and both endpoints of every exchange — agrees exactly.

Completeness. The fix required all *seven* `joffset` sites, found in two passes. The first (`89904070`) covered the sender and unpack sides in `shearing_box_cc.cpp` and `shearing_box_fc.cpp`. The second (`ca63f727`) covered `shearing_box_tasks.cpp`, which carries its *own* copy of the case-1/2/3 logic to post the `MPI_Irecv`s (`InitRecv`, `ClearRecv`, `ClearSend`) with its own per-block `dx2`. That the sender/recv-poster disagreement is deterministic for a given mesh and $y_{\rm shear}$ is precisely why the crash reproduced at the identical cycle in both attempts.

A sweep of the tree confirmed no other sites: FARGO's `joffset` is sender-local (pack-only, fixed-size buffers behind the I1 guard) and the FFT and multigrid solver shifts already derive from global quantities. Remap ϵ values shape content only, never message sizes, so last-ulp differences there are harmless.

9 Input-file interface

9.1 Parameters

Block	Parameter	Default	Meaning
`<shearing_box>`	`qshear`	—	q
	`omega0`	—	Ω_0
	`orbital_advection`	`true`	`false` selects the full-velocity (non-FARGO) frame and lifts I2
`<gravity>`	`solver`	—	`multigrid` or `fft`
	`four_pi_G`	—	$4\pi G$
	`mg_bc`	`none`	`slab`: open (vacuum) x_3 boundaries (Phase 3b); also `zerofixed`, `zerograd`, `multipole`
	`mg_remap`	`plm`	Remap order for the shear ghost fill and the slab plane roll/rings
	`mg_nghost`	1	Must be 1 with shear or slab
	`mg_fc_interp`	`linear`	`quadratic`: second-order level-jump ghosts on the finest level (§5.5); input file only
	`root_on_host`	`false`	Must be false with shear or slab
	`threshold`	—	< 0 selects fixed-iteration mode
	`niteration`	—	Cycle count in fixed mode
	`vert_bc`	`periodic`	FFT solver only: `open` = KO09 two-solve vacuum x_3
`<hydro_srcterms>`	`beta_cooling`	`false`	$\dot{U} = -U\Omega_0/\beta$ (Phase 3a)
	`bcool_beta`	—	$\beta = \Omega_0 t_{\rm cool}$
	`thermal_cooling`	`false`	$t_{\rm cool} = b(\rho/P)^3$ (SC14 eq. 8)
	`tcool_b`	—	b
	`cool_eps`	0.02	Timestep cap $\Delta t \le \epsilon\, t_{\rm cool}$
`<problem>`	`amr_moving_ring`	`false`	Enroll the prescribed moving-ring criterion
	`ring_xc0`, `ring_xamp`	0	Ring center and oscillation amplitude
	`ring_freq`	1	Ring oscillation frequency
	`ring_hwidth`	—	Ring half-width
`<mesh>`	`ix1_bc`, `ox1_bc`	—	`shear_periodic` (both, or neither)
`<mesh_refinement>`	`refinement`	—	`static` or `adaptive`
	`max_nmb_per_rank`	—	Needs transient headroom under moving-ring AMR

9.2 Reference inputs

File (under `inputs/`)	Configuration
Phase 0 — hydro only	
`shearing_box/hydro_incompress_shwave_smr.athinput`	SMR, no FARGO
`shearing_box/hydro_incompress_shwave_smr_ring.athinput`	Annular ring, FARGO
`shearing_box/hydro_incompress_shwave_smr_ring2.athinput`	Two nested rings
`shearing_box/hydro_incompress_shwave_smr_bndry{,2}.athinput`	Refined boundary faces (I1)
`shearing_box/hydro_incompress_shwave_amr_ring.athinput`	Moving ring, AMR
`shearing_box/epicycle_smr_bndry.athinput`	Epicycle, refined boundary
`shearing_box/epicycle_amr_ring{,2,_nofargo}.athinput`	Epicycle under AMR
Phases 1–2 — with gravity	
`tests/fft_poisson_shear.athinput`	Static Poisson, uniform (FFT/MG A/B)
`tests/mg_poisson_shear_smr.athinput`	Static Poisson, interior ring (2a)
`tests/mg_poisson_shear_bndry.athinput`	Static Poisson, boundary annuli (2b)
`tests/swing_selfgrav.athinput`	Swing amplification, uniform
`tests/swing_selfgrav_smr.athinput`	Swing, interior ring (2a)
`tests/swing_selfgrav_bndry.athinput`	Swing, boundary annuli (2b)
`tests/swing_selfgrav_amr.athinput`	Swing, moving ring, AMR (2c)
`tests/ts41_sin3.athinput`, `tests/ts43_jeans.athinput`	Tomida & Stone (2023) §4 consistency
Phase 3 — stratified (slab-open x_3)	
`tests/mg_poisson_slab.athinput`	Sheet test, uniform, periodic x_1
`tests/mg_poisson_slab_shear.athinput`	Sheet test, sheared, FFT cross-check
`tests/mg_poisson_slab_smr.athinput`	Sheet test, level-1 midplane slab
`shearing_box/gravito_turb_sc14.athinput`	SC14 $\beta = 10$ production setup

Because `solver` is a single input key, most gravity inputs are a one-override A/B against the FFT oracle: adding `-gravity/solver=fft` on the command line runs the identical problem through the independent solver.

9.3 Building

```
cmake -B build -D Athena_ENABLE_MPI=ON     # add -D Kokkos_ENABLE_CUDA=ON for GPU
cmake --build build -j
```

`multigrid/multigrid_shear.cpp` is registered in `src/CMakeLists.txt`; the MPI seam of §4.9 compiles out entirely in serial builds, so serial and MPI builds produce identical output on identical problems.

10 Guard reference

Every fatal condition introduced by this work, with the mechanism it protects.

Location	Condition	Protects
`Mesh::CheckShearingBoxRefinement`	x_1-face blocks span more than one level	I1: same-level wrap and same-size blocks along each face

Location	Condition	Protects
`Mesh::CheckShearingBox-Refinement`	Incomplete azimuthal ring, FARGO on	I2: same-level, same-size x_2-face neighbors for the per-level shift
`OrbitalAdvectionCC::-RecvAndUnpackCC`	Actual shift > `maxjshift`	Buffer depth; converts an out-of-bounds read into a diagnosable abort
`MHD::MHD`	FARGO + refinement	Face-centered B remap at fine/coarse interfaces ($\nabla\cdot B$) is unanalyzed
`MGGravityDriver`	x_2 or x_3 not periodic	Plane wrap assumes periodic azimuth/vertical
`MGGravityDriver`	`mg_bc` set with shear	Conflicting boundary specification
`MGGravityDriver`	`root_on_host` with shear	Root fill is a device kernel
`MGGravityDriver`	`mg_nghost > 1` with shear	Untested; octet fill assumes one ghost
`MGGravityDriver`	Unrecognized `mg_remap`	Typo protection
`MGGravityDriver`	x_1-face blocks span more than one level	Plane sizing (`shear_reflev_`) and octet face tiling
`MGGravityDriver`	`mg_bc = slab` on a non-3D mesh, or without (shear-)periodic x_1 / periodic x_2, or on only one x_3 face	Slab planes assume a horizontally wrapped box with both x_3 faces open
`MGGravityDriver`	`mg_bc = slab` without an FFT build	Plane computation needs kokkos-fft
`MultigridDriver::Check-SlabBlockLevels`	x_3-face block above root level	Plane pyramid indexing; re-checked after every AMR remesh
`MultigridDriver::Apply-PhysicalBoundariesOctet`	Octet on a slab x_3 boundary	Unreachable under the invariant above; defensive

11 Known limitations and deferred work

1. **Mixed-level shear boundaries.** I1 forbids them. Supporting them means combining the y-shift with prolongation *inside* the shear communication, for both the hydro exchange and the multigrid fill.

2. **MHD orbital advection with refinement.** Fatal. The face-centered B remap at fine/coarse interfaces needs a $\nabla\cdot B$ analysis before the guard can be lifted. MHD without orbital advection, and hydro with it, are both supported.

3. **`root_on_host` with shear.** Fatal. `RootShearBoundaryX1` is a device kernel; a host twin (analogous to the scalar remap twins of §5.2) would lift this.

4. **`mg_nghost > 1` with shear.** Fatal. The block fill is written for general `ngh`, but `FillShearOctetGhosts` assumes `ngh` $= 1$ because octet interiors are two cells deep.

5. **Redundant remap work under MPI.** Every rank remaps the whole plane after the `Allreduce`. This is what makes multi-rank results bit-identical to serial ones, and the planes are small relative to the volume data, but it is redundant work that a partitioned remap would avoid — at the cost of that guarantee.

6. **The stopping rule on refined meshes.** Stagnation-based termination (`threshold = 0`) can hit the cycle cap mid-run on the wound-up $k_x \sim 0$ mode, whose coarse-grid correction crosses the sheared x_1 wrap. Refined shearing-box inputs therefore default to FMG with a fixed iteration count. This is a practical convergence-rate issue, not a correctness one — the fixed point is unaffected — but it is the one place where the sheared identification visibly degrades multigrid efficiency.

7. **The $O(\Delta t)$ phase offset.** Gravity freezes the shear phase at `pmesh->time`; hydro's shear-periodic BC uses `time+dt` on the final RK stage. This is a pre-existing convention retained deliberately so the multigrid and FFT solvers remain directly comparable; revisiting it means revisiting both solvers and their validation together.

8. **Slab boundaries require root-level x_3-face blocks.** The plane pyramid could in principle start at a uniformly refined boundary level (as the shear planes do, §5.2), but no target problem needs it; the guard keeps the mapping simple.

9. **The full-density Allreduce.** `ComputeSlabPlanes` reduces the whole root-resolution density array (∼25 MB at SC14's standard resolution) once per solve, in exchange for bit-identical planes on every rank. A partial-spectrum reduction (per-rank slice FFTs, reduce only the two face spectra) would cut this to ∼1 MB if profiling ever demands it.

10. **The force at level jumps is first order by default.** With the default `mg_fc_interp = linear` the force next to a level jump is first order (0.43% of the peak force at a 256^2 root on the Poisson test); `quadratic` (§5.5) makes it second order (0.035%, as R. Li's composite solver). The default stays linear for bit-for-bit continuity of existing runs. In quadratic mode the composite operator is nonsymmetric: fine for the red-black FAS V-cycle, but a symmetric Krylov wrapper would need care. Not yet tested in quadratic mode: GPU builds, restarts, dust self-gravity runs.

11. **The $k_\perp = 0$ interface offset.** On refined meshes the conservative level interface leaves a constant $O(h^2)$ potential offset inside enclosed refined regions (§6.3). It is force-free and converges at second order, but it means refined-region potentials should be compared gradient-wise, not value-wise.

12. **Density-floor mass injection.** With `diode` vertical boundaries and `dfloor` $= 10^{-4}$, the box gains mass at the floor rate (∼$0.1\%/10\,\Omega_0^{-1}$ in the hold test). SC14 ran the same floors; the effect is negligible against their ∼10% transient mass loss, but exact mass budgets should account for it.